

Optimized Container Orchestration for Web Applications

2º DAW – Despliegue de Aplicaciones Web | RA3

Date: May 2026

Student: María Abascal Jiménez

Repository: <https://github.com/mabajim065/docker-web-app-ra3>

1. Deployment Manual

a. Description of the multitier architecture.

I have implemented a three-tier architecture deployed with Docker and Docker Compose divided into 4 parts:

1. Nginx: It is the only service visible from the outside. It functions as a reverse proxy, redirecting traffic to the frontend or the backend depending on the URL.
2. Frontend: User interface isolated in its own container. It only communicates with the backend through the proxy, in my case react.
3. Backend: Manages the business logic and database calls. It is only accessible internally through the Docker network, in my case spring boot.
4. Database: Only responds to backend requests and is protected from external or frontend access.

b. Explanation of the role of each service.

1. Nginx: Only external entry. Directs traffic: / to the Front and /api to the Back.
2. Frontend: User interface (React). Requests data from the Back through Nginx, it is the visible part.
3. Backend: Logic (Spring Boot). Processes requests and queries the database.
4. MySQL: It is where data is stored. Only the Backend has the key.

c. Steps to deploy the environment using Docker Compose.

1. Have Docker Desktop installed and running and Git installed.
2. Clone the repository with git clone and put your url and enter the folder with cd docker-web-app-ra3.
3. Create the environment variables file by copying the example with copy .env.example .env and edit it with the desired credentials.
4. Build and start all services with docker compose up -d --build. Verify that all services are running with docker compose ps.
5. All must appear as Up and the db container must show the state (healthy).

6. Open the browser and access `http://localhost` to check that the application works correctly.
7. To stop the services use `docker compose down`, or `docker compose down -v` if you also want to remove the volumes.
8. To update a specific service use `docker compose build backend` followed by `docker compose up -d backend`.

d. Description of the Configuration Files and the Network

The most important files are:

- `docker-compose.yml`: Defines the 4 services, the networks, the data volume, resource limits, and the MySQL healthcheck.
- `.env`: Stores the database passwords. It is not uploaded to GitHub.
- `nginx/nginx.conf`: Distributes the traffic: `/` goes to the frontend and `/api` goes to the backend.
- `frontend/Dockerfile` and `backend/Dockerfile`: Instructions to build the images using multi-stage builds.
- `db_backup.sh`: Script for creating database backups.

e. Instructions for Starting, Stopping, and Updating Services

1. Start all services: `docker compose up -d --build`
2. View the status of the services: `docker compose ps`
3. View the logs of a service: `docker compose logs backend`
4. Stop all services: `docker compose down`
5. Stop and also delete the data: `docker compose down -v`
6. Restart a service: `docker compose restart backend`
7. Update a specific service: `docker compose build backend` and then `docker compose up -d backend`

f. How Multi-Stage Builds Reduce Image Size and Improve Security

A multi-stage build divides the Dockerfile into two phases, the first phase where the project is compiled using heavy tools like Node.js or Maven, and the second phase where only the final result is copied to a lightweight image based on Alpine. This ensures that the final image does not contain source code or compilation tools, making it smaller and more secure.

g. How the Healthcheck Prevents Connection Errors During Initial Deployment

MySQL takes several seconds to initialize upon startup. Without a healthcheck, the backend would attempt to connect to the database before it was ready, causing connection errors and making the application fail. To prevent this from happening, a healthcheck was configured in the MySQL container that checks every 10 seconds if the database is ready. The backend has a `depends_on` configured with condition: `service_healthy`, which makes Docker wait for the MySQL healthcheck to be successful before starting the backend, guaranteeing that the connection is always established correctly.

2. Administration Manual

a. Procedures to manage deployed services.

The main commands to manage the services are, where it says name is the name of any of the services:

- View the status of the containers: `docker compose ps`
- View the logs of a service: `docker compose logs name`
- Restart a service: `docker compose restart name`
- Stop all services: `docker compose down`
- Monitor resource usage: `docker stats`
- Update a service: `docker compose build name` and then `docker compose up -d name`

b. Security considerations and applied measures.

- Minimum exposed ports: Only Nginx exposes port 80 to the host. The rest of the services are only accessible from the internal Docker network.
- Environment variables: Database passwords and credentials are saved in an `.env` file that is not uploaded to GitHub, so that sensitive information does not leak into the repository.
- Users without root privileges: Both the frontend and backend create their own user inside the container and run the application with that user, avoiding running processes as root.
- Isolated networks: Two separate internal networks are used. The database is only accessible from the backend, not from the frontend or Nginx.
- Fixed image versions: All images use specific versions to avoid unexpected changes when updating.
- Resource limits: Memory (512MB) and CPU (0.5) limits have been defined for the backend and the database, so that a failure does not take all the host's resources.

c. Maintenance and scalability recommendations

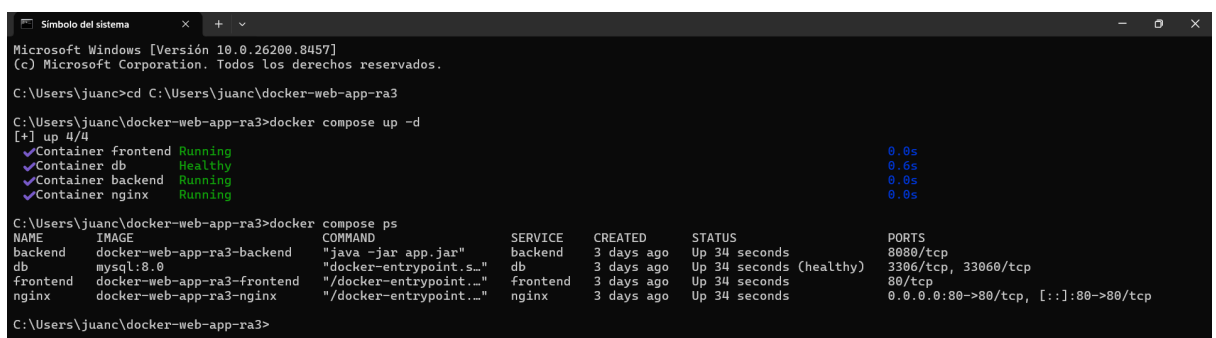
- Backups: Run the db_backup.sh script regularly and save the .sql files outside the server.
- Monitoring: Use docker stats to monitor the CPU and memory consumption of the containers.
- Image updating: Check from time to time if there are newer versions of the base images to receive security patches.
- Scalability: If the application needs to support more traffic, the number of backend containers can be increased with docker compose up -d --scale backend=2.
- Production: For a real environment, it is recommended to add HTTPS with an SSL certificate, use stronger passwords, and consider Docker Swarm or Kubernetes for greater availability.

d. Summary of the tests performed and results obtained.

Several tests were performed to check the operation of the deployment. First, it was verified that the frontend was accessible from the browser by entering http://localhost, loading the application fine from Nginx. It was also verified that the reverse proxy correctly routed requests to /api towards the backend. With docker compose logs backend, it was confirmed that the backend connected to the database without connection errors. With docker compose ps, it was verified that the database showed the status as healthy and that only Nginx exposed port 80 to the host, leaving the rest of the services to the internal network. Finally, the db_backup.sh script was executed, which correctly generated the .sql file with the backup in the ./backups folder.

3.Screenshots

1. Container status



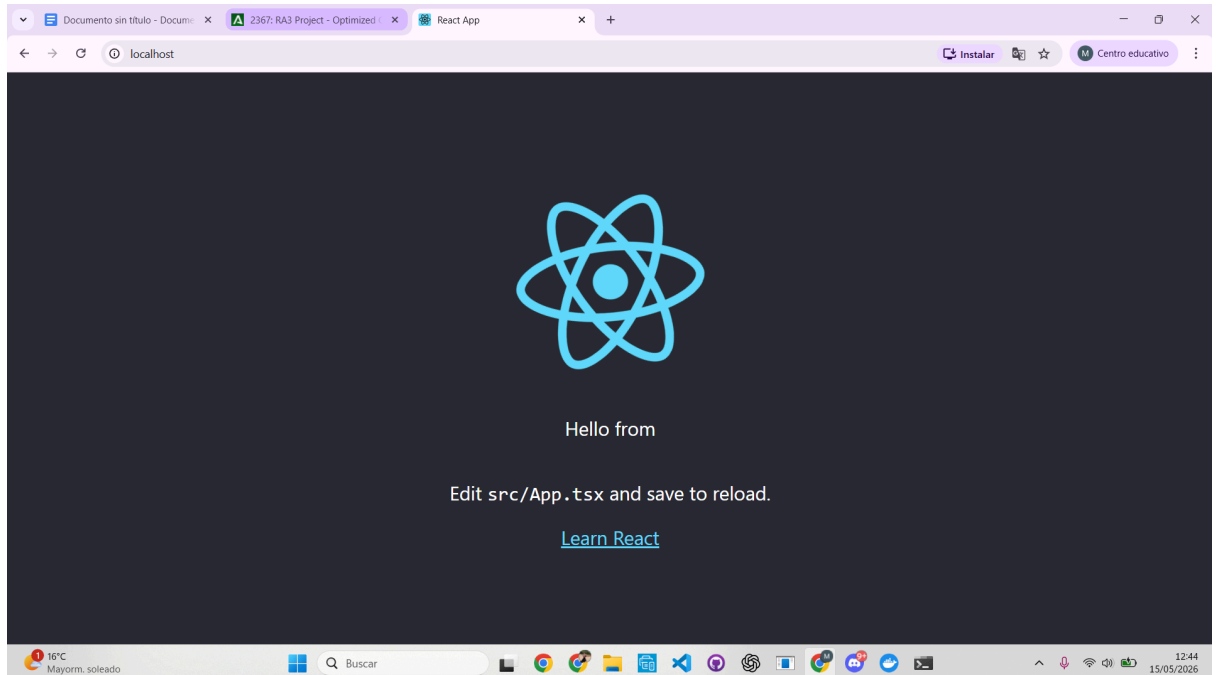
```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26280.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\juanc>cd C:\Users\juanc\docker-web-app-ra3
C:\Users\juanc\docker-web-app-ra3>docker compose up -d
[+] up 4/4
✓Container frontend Running 0.0s
✓Container db Healthy 0.6s
✓Container backend Running 0.0s
✓Container nginx Running 0.0s

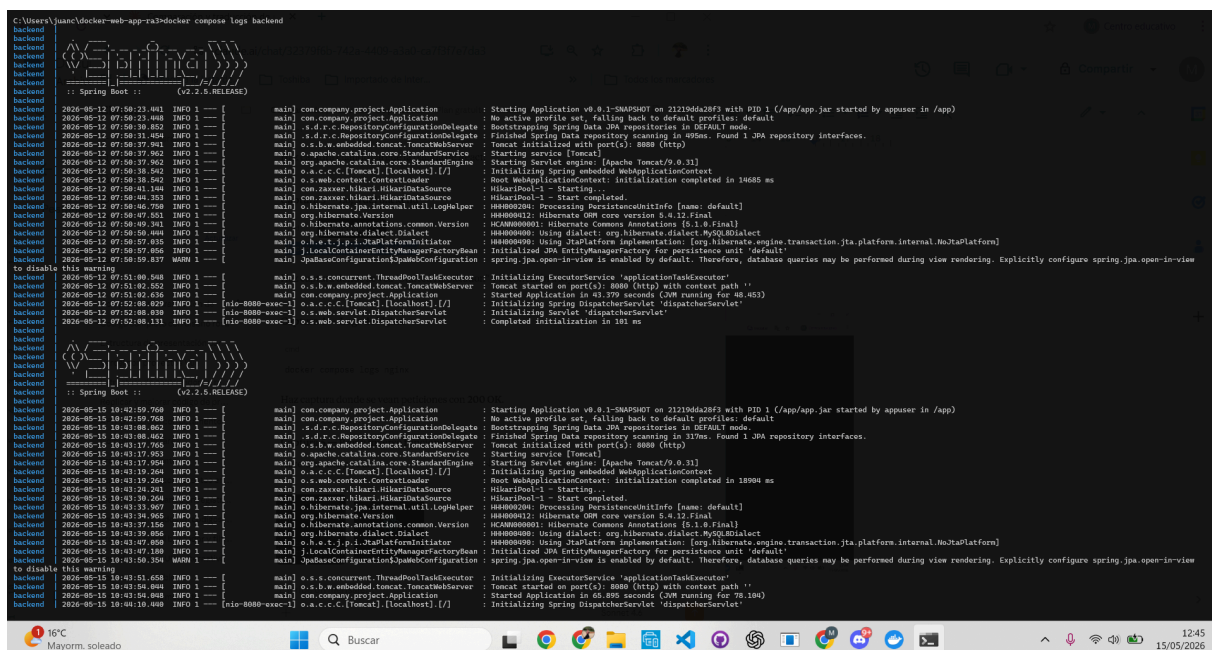
C:\Users\juanc\docker-web-app-ra3>docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
backend docker-web-app-ra3-backend "java -jar app.jar" backend 3 days ago Up 34 seconds 8080/tcp
db mysql:8.0 "docker-entrypoint.s..." db 3 days ago Up 34 seconds (healthy) 3306/tcp, 33060/tcp
frontend docker-web-app-ra3-frontend "/docker-entrypoint..." frontend 3 days ago Up 34 seconds 80/tcp
nginx docker-web-app-ra3-nginx "/docker-entrypoint..." nginx 3 days ago Up 34 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp

C:\Users\juanc\docker-web-app-ra3>
```

2. Frontend in the browser



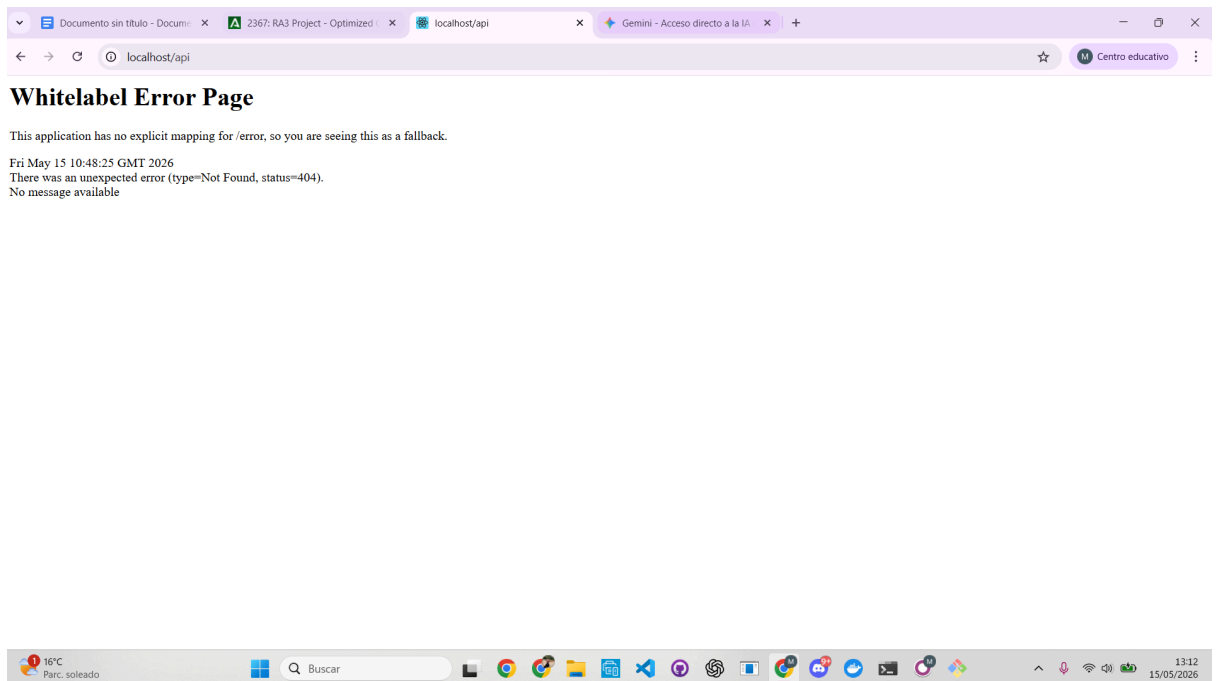
3. Backend logs



4. Nginx logs

```
Simbolo del sistema
C:\Users\juanc\docker-web-app-ra3\docker compose logs nginx
nginx /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-tcp-by-default.sh
nginx 10-listen-on-tcp-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
nginx /docker-entrypoint.sh: Configuration complete; ready for start up
nginx 2026/05/12 07:50:14 [notice] 181: using the "epoll" event method
nginx 2026/05/12 07:50:14 [notice] 181: built by gcc 5.2.0 (Alpine 5.2.0)
nginx 2026/05/12 07:50:14 [notice] 181: OS: Linux 6.6.87.2-microsoft-standard-WSL2
nginx 2026/05/12 07:50:14 [notice] 181: getrlimit(RLIMIT_NOFILE): 1048576:1048576
nginx 2026/05/12 07:50:14 [notice] 181: start worker processes
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 30
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 31
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 32
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 33
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 34
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 35
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 36
nginx 2026/05/12 07:50:14 [notice] 181: start worker process 37
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET / HTTP/1.1" 200 608 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /static/css/main.977c9b0a.css HTTP/1.1" 200 1800 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /mainfest.js HTTP/1.1" 200 517 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /static/js/main.0be4f93e.js HTTP/1.1" 200 13232 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /static/media/logo-2627ead7c18777cc33600c18c2076728.svg HTTP/1.1" 200 2078 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /favicon.ico HTTP/1.1" 200 3158 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /logo192.png HTTP/1.1" 200 5307 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:07:52:07 +0000] "GET /api HTTP/1.1" 404 135 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:53 +0000] "GET /api/messages HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:53 +0000] "GET /api HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:53 +0000] "GET /api HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:54 +0000] "GET /api HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:54 +0000] "GET /api HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [12/May/2026:08:00:54 +0000] "GET /api HTTP/1.1" 404 306 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-tcp-by-default.sh
nginx 10-listen-on-tcp-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
nginx /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
nginx /docker-entrypoint.sh: Configuration complete; ready for start up
nginx 2026/05/12 10:42:35 [notice] 181: using the "epoll" event method
nginx 2026/05/12 10:42:35 [notice] 181: built by gcc 5.2.0 (Alpine 5.2.0)
nginx 2026/05/12 10:42:35 [notice] 181: OS: Linux 6.6.87.2-microsoft-standard-WSL2
nginx 2026/05/12 10:42:35 [notice] 181: getrlimit(RLIMIT_NOFILE): 1048576:1048576
nginx 2026/05/12 10:42:35 [notice] 181: start worker processes
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 29
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 30
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 31
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 32
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 33
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 34
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 35
nginx 2026/05/12 10:42:35 [notice] 181: start worker process 36
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET / HTTP/1.1" 308 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET /static/css/main.977c9b0a.css HTTP/1.1" 308 0 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET /static/js/main.0be4f93e.js HTTP/1.1" 308 0 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET /static/media/logo-2627ead7c18777cc33600c18c2076728.svg HTTP/1.1" 308 0 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET /favicon.ico HTTP/1.1" 308 0 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
nginx 172.23.0.1 - - [15/May/2026:10:48:09 +0000] "GET /logo192.png HTTP/1.1" 308 0 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/107.0.0.0 Safari/537.36" "-"
```

5. Reverse proxy working



6. Backup script

```
MINGW64:/c/Users/juanc/docker-web-app-ra3

juanc@LAPTOP-OVILC63U MINGW64 ~
$ cd /c/Users/juanc/docker-web-app-ra3

juanc@LAPTOP-OVILC63U MINGW64 ~/docker-web-app-ra3 (master)
$ bash db_backup.sh
Starting backup of database 'appdb'...
mysqldump: [warning] Using a password on the command line interface can be insecure.
Backup completed successfully: ./backups/backup_20260515_131027.sql

juanc@LAPTOP-OVILC63U MINGW64 ~/docker-web-app-ra3 (master)
$ |
```